

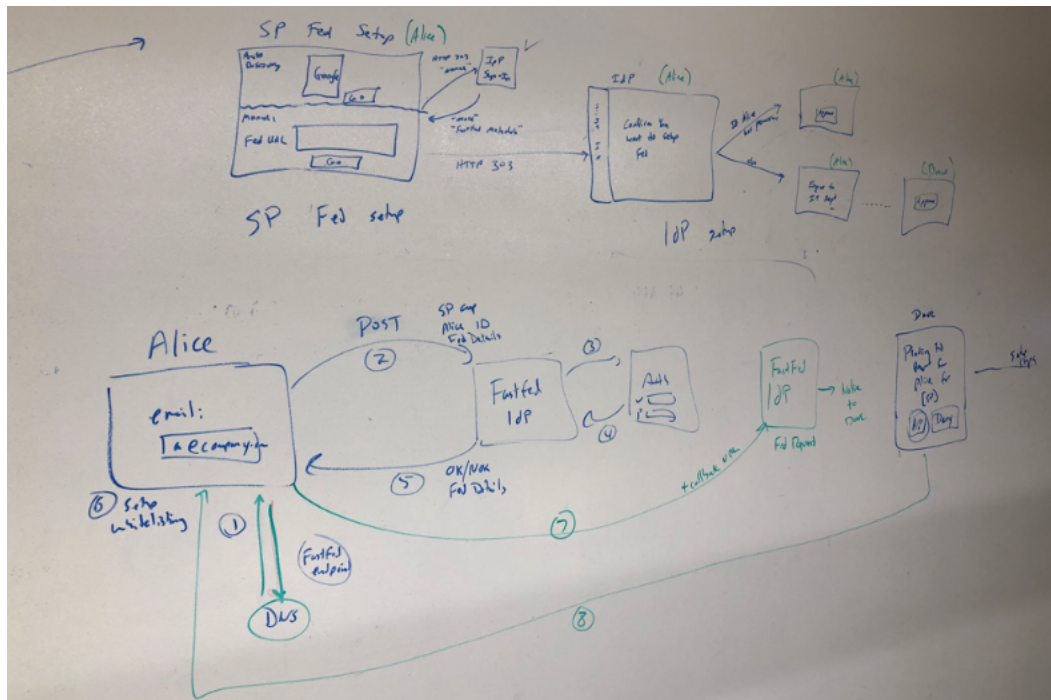
Information Disclosure Risks from FastFed Discovery

During a review of the FastFed specification, it was pointed out that FastFed Discovery allows any actor to iterate through well-known domain names (such as all companies in the S&P 500) and inspect the Identity Provider they are using as well as information such as the TenantID in that provider and protocol preferences for SSO and Provisioning.

An alternative flow was explored to minimize publicly accessible information during discovery. This began as a whiteboard sketch. Subsequently, a deeper analysis was performed. This document presents the results of the deeper analysis.

See Appendix A for detailed step-by-step flows, including security/phishing risks at various steps.

Figure 1: Whiteboard sketch showing original flow (top) and alternative flow (bottom)



Summary of Results

Both flows accomplish similar results. The net difference is the alternative flow requires a few extra redirects and endpoints. As a natural side-effect, it introduces 2 new vectors for security risks that arise from the additional actions (see Steps #10 and #11 in the alternative flow), plus additional implementation effort.

Depending on implementation details, there is a potential adjustment to the customer experience in that Alice, the administrator of the service, could be prompted to authenticate twice at the SP. (See Step #1 and Step #10.)

Finally, the requirement that the SP administrator also be authenticatable with the IdP shuts the door to implementing [FastFed Scenario #3](#) as currently written. That may-or-may-not be OK, since the group has not deeply discussed or vetted this scenario.

The decision largely comes down to a judgement call on whether the data protections are sufficiently beneficial to justify the drawbacks. There is also a meta-question on whether FastFed Discovery will actually be used in practice.

The remainder of this document summarizes the risks and benefits in order to inform a judgement call.

General Headwinds Against Discovery

Before considering a variety of discovery mechanisms, it is worth examining the meta-question of whether discovery itself (in any form) would be acceptable to users. There are forces that will discourage its usage, making it possible that we are contemplating a feature destined to have no real-world usage, regardless of how implemented.

To evaluate this question, we can start by recognizing that despite all best-efforts to strip unnecessary information from public discovery flows, the act of discovery itself makes it necessary to discover *something*.

The something, at minimum, is a URL where the end-user may be redirected to authenticate and consent to the release of additional information.

For all practical purposes, this URL will be an endpoint hosted by the Identity Provider. This could be obfuscated through CNAMEs, but a determined explorer can resolve those names and make a well-informed decision about the underlying Identity Provider.

The result is that, despite all best efforts to minimize discoverable information, it inevitably remains possible to learn the Identity Provider for a given organization that has enabled FastFed Discovery.

A creative blogger could use this capability to run a weekly scan, across a wide range of known domains, and publish a scorecard showing adoption rates of cloud IdPs. One could imagine

categories such as penetration into the S&P 100, 500, etc. The report could also include “Recent Moves”, showing organizations who recently switched IdPs.

This detail of visibility could be uncomfortable for both customers and identity providers, outweighing the convenience of discovery.

At this point, we don’t have data on the market reception to this risk. Perhaps organizations don’t care and love the convenience. Perhaps the opposite happens. From an implementation perspective, we can proceed but should recognize the adoption risks. Pre-vetting the idea with customers and local business leaders may help us understand the likelihood of adoption.

Sensitivity of FastFed Metadata

For the sake of argument, if we presume that discovery remains a valid feature despite the unavoidable data leakage, the next question is how much discovery information to expose. As currently written, the specification allows any public entity to retrieve the FastFed metadata for an organization that consists of the following:

- The IdP Name (e.g. GSuite)
- The IdP TenantID (e.g. Often a GUID like “C043qg4cj”)
- Icons & Logos for the IdP
- The protocols supported by the IdP, such as SAML/OIDC/SCIM
- URL to initiate the FastFed Handshake with the IdP

The proposed alternative strips this metadata down to the following

- The IdP Name (e.g. GSuite)
- Icons & Logos for the IdP
- URL to initiate discovery of remaining metadata from the IdP

The first two items are retained to maintain usability (See step #3 in the alternative flow.) The last URL is basically a swap with the original FastFed Handshake URL.

The hidden items are:

- The IdP TenantID (e.g. Often a GUID like “C043qg4cj”)
- The protocols supported by the IdP, such as SAML/OIDC/SCIM

The sensitivity of the latter is likely minimal. Knowing that a customer has chosen to use GSuite, for example, it is likely unsurprising that GSuite supports SAML/OIDC/SCIM. This information is potentially in the public documentation of the provider.

This leaves the TenantID as the remaining piece of potentially sensitive information which is hidden by the alternative discovery flows. The sensitivity of this value is an open question. TenantID is often part of the Sign-In URL vended to applications for SSO purposes. For example:

```
https://accounts.google.com/o/saml2/idp?idpid=C043qg4cj
```

While not technically public, TenantIDs are distributed widely without expectation of secrecy and convey no privileges on their own.

Therefore, the value of hiding the metadata boils down to the value of hiding the TenantID from public discovery, and whether the benefit is sufficient when weighed against the implementation costs.

Alternatives to Discovery

Up to now, this document has considered the risks of implementing automated discovery and the options for minimizing discoverable information.

A remaining question is what happens if customers prefer to disable discovery. What is the alternative experience?

One possible option, briefly discussed in the whiteboarding session, is the notion of an IdP-initiated flow. In this experience, Alice starts at her IdP and clicks a button to request adding a new application to her catalog. The IdP may verify that Alice is permitted to do so, and then generates a FastFed URL that she may copy-and-paste into the SP to start the FastFed handshake. This IdP could incorporate parameters into the URL such as a nonce that constrain it to a single use only by Alice, or an expiration date. The implementation details are outside the scope of FastFed but are possible when an IdP generates on-demand URLs.

The IdP could also maintain a whitelist of known valid SPs and restrict the vended FastFed URL to usage only with a particular SP. In some cases, auto-approval could be allowed for known safe SPs. Again, these details are outside the scope of FastFed but remain valid implementation options.

In addition, these approaches mitigate some of the phishing and security risks that arise due to automated discovery.

The FastFed specification as currently written appears to support these alternative scenarios. It is an implementation choice of the IdP on which features to enable for their customers.

Auxiliary Benefit of the Alternative Flow

The alternative flow was initially proposed to alleviate a discovery concern, but in the process, it introduced a new feature.

Specifically, in Step #13, the SP is given a link which can be followed to view the pending approval status in the IdP. The SP can display this link to Alice. This solves a usability gap in the original spec in which Alice is redirected from the SP to the IdP to submit the request, pending approval by her IT department. Alice may not remember where she was redirected, nor have it bookmarked. When she wants to check the status of her request, she may not remember where to go.

If Alice started the flow at the SP, it would be nice if the SP could help Alice find this information again. One way to achieve this, without introducing more features into the spec, is to allow the SP to redirect Alice (again) to the same FastFed Handshake URL that started the original flow as per section 6.2.1.6.1 of the specification. Duplicate detection in this handler can detect that Alice already has a pending request for the SP and display the current status, rather than initiating a new registration. This duplicate detection is likely necessary regardless, and could provide a convenient channel for the SP to help Alice check the status of a pending registration.

The feature is worth considering, independent of the decision around automated discovery.

Appendix A) Step-by-Step Flows

Original Flow (summarized from existing spec)

1. Alice visits the SP, authenticates, and navigates to a UI page for configuring SSO.
2. Behind the scenes, the SP attempts to discover Alice's IdP by querying a well-known subdomain derived from her email address. The discovery process, if successful, results in a publicly-accessible pointer to the FastFed Metadata for the IdP.
3. If discovery is successful, Alice sees her IdP Name and Icon in the SP. She can click a button to consent & continue.

Security Risk -> The discovery endpoint must use TLS to demonstrate ownership of the domain. However, if the SP fails to validate the TLS certificate correctly, or some other breakdown occurs in the Certificate Authority so that certificates are attained by a malicious actor, then Alice can be tricked into enabling someone else's IdP to federate to her application.

4. Behind the scenes, the SP whitelists the IdP.
5. SP issues an HTTP 303 to send Alice to the IdP. The redirect message includes a URL where the IdP can retrieve the FastFed Metadata for the SP.

Security Risk -> Continuing the same risk as Step 3, the malicious actor could also harvest Alice's legitimate corporate credentials by tricking her into signing in to the imposter IdP.

6. The IdP authenticates Alice and verifies she is allowed to submit requests for new SSO configurations.
7. The IdP loads the FastFed Metadata for the SP and verifies compatibility.
8. Alice sees the Name & Icon for the SP. She is asked to confirm before proceeding. If Alice is not authorized to approve the SSO configuration herself, it may be enqueued for subsequent approval by another team.

Security Risk -> If Alice were tricked into visiting an imposter instance of her SP in step 1, and didn't realize it sooner, she would have no way to detect it here either. Alice would agree to setup user provisioning and SSO into the imposter app.

9. When the request is finally approved, the IdP makes a callback to the SP using the callback URL specified in the service provider's FastFed Metadata. At this point, the

remainder of the flow is the same as the alternative (below) and follows the FastFed specification starting at section 6.2.3.

Alternative Flow (interpreted from the whiteboard sketch)

1. Alice visits the SP, authenticates, and navigates to a UI page for configuring SSO.
2. Behind the scenes, the SP attempts to discover Alice's IdP by querying a well-known subdomain derived from her email address. In order to minimize information disclosure, the Discovery endpoint returns only the IdP displayName, icons, and a URL for continuing the handshake.
3. If discovery is successful, Alice sees her IdP Name and Icon in the SP. She can click a button to consent & continue.

Security Risk (same as original flow) -> The discovery endpoint must use TLS to demonstrate ownership of the domain. However, if the SP fails to validate the TLS certificate correctly, or some other breakdown occurs in the Certificate Authority so that certificates are attained by a malicious actor, then Alice can be tricked into enabling someone else's IdP to federate to her application.

4. Alice clicks the button and an HTTP 303 sends Alice to the IdP endpoint. The redirect parameters include Alice's email address, a URL to retrieve the SP's FastFed Metadata, and another URL to redirect Alice back to the SP later in the process.
5. Alice is prompted to authenticate to her IdP. The IdP may also apply additional authorization rules at this time regarding whether Alice is permitted to initiate new federations. This is an implementation detail outside the scope of FastFed.

Security Risk (same as original flow) -> Continuing the same risk as Step 3, the malicious actor could also harvest Alice's legitimate corporate credentials by tricking her into signing in to the imposter IdP.

6. The IdP validates compatibility with the IdP and signals an error if not compatible.
7. In the IdP, Alice sees the Name & Icon for the SP. She is asked to confirm before proceeding.

Security Risk (same as original flow) -> If Alice were tricked into visiting an imposter instance of her SP in step 1, and didn't realize it sooner, she would have no way to detect it here either. Alice would agree to setup user provisioning and SSO into the imposter app.

8. After attaining Alice's consent, the IdP issues an HTTP 303 and sends Alice back to her SP using the redirect endpoint provided in step #4. The redirect parameters include a URL to retrieve the FastFed metadata for the IdP, plus a callback URL. The callback URL may include a signature, expiration data, and/or nonce to limit the blast impact if a malicious interceptor captured the redirect message. (See Security Risk in step 10.)
9. The SP receives the redirect message.

10. The SP whitelists the IdP using the FastFed metadata provided in the redirect message.

Security Risk (new) -> The SP needs to verify that the redirect message is a legitimate continuation of a flow that Alice previously initiated, as opposed to a malicious actor who is spraying these messages at SPs and trying to trick them into whitelisting a malicious IdP. The verification process could include forcing Alice to reauthenticate and re-consent. Alternatively, verification could leverage some form of signature/nonce in the callback URL that was issued at step #4, although this presents the risk of the callback URL being intercepted and used by a malicious actor to register their own IdP.

11. The SP makes a callback to the IdP using the callback URL received in Step #8. This “enqueues” the request for official approval within the IdP. As part of this request, the SP provides another callback endpoint that the IdP will invoke after approval is granted.
Security Risk (new) -> The IdP needs to verify that the callback is a legitimate continuation of a flow that Alice previously initiated, as opposed to a malicious actor who is spraying these messages at IdPs and trying to trick them into whitelisting a malicious SP. One protection is that the callback URL could be hard to guess and only discoverable after Alice consents to its release in Step #7 above. This reduces the risk, but doesn't protect against the interception of the callback URL during the redirect flows via PKCE-style risks in mobile environments. Verification could leverage some form of signature/nonce in the callback URL that was issued at step #8, but again, it doesn't protect against the callback URL being intercepted.
12. If Alice is authorized to create new SSO relationships in the IdP, her request may be automatically approved instead of being queued for approval by others.
13. The IdP responds to the SP. The response may include a URL where Alice can view the status of her request at the IdP. The SP can use this information to display to Alice the current status and include the URL so Alice knows what's happening and how to check the status of her request at the IdP.
14. When the request is finally approved at the IdP, the IdP makes a callback to the SP using the callback URL specified in Step #11. At this point, the remainder of the flow is the same as the existing FastFed specification starting at section 6.2.3.